26021 RTOS5

Mastering USB communication: Develop USB Device applications with Zephyr's USB Device Stack on your custom board

26021 RTOS5 - Mastering USB communication: Develop USB Device applications with Zephyr's USB Device Stack on your custom board

Introduction

USB is now a standard serial communication channel to connect embedded systems to PCs. Upon completion of the course, you will be able to create an enumerable USB device and transfer data between a Microchip MCU and a PC using Zephyr's USB Device Stack. The hands-on part of this class is based on a development board that is not yet supported by Zephyr tree, and you will also learn how to add your own board to a Zephyr based application.

Upon completion, you will be able to:

- **Create your custom board support and compile the Zephyr blinky example on it.**
- **Add USB CDC ACM class to the example and communicate with the PC using a terminal.**
- **Transfer generic data between MCU and PC applications.**

This class includes four labs:

Lab 1 - Run the Blinky sample project on your custom board

Lab 2 - Redirect the Console to the USB

Lab 3 - Build a CDC device and communicate with a terminal on PC

Lab 4 - Implement a simple command-response protocol

Prerequisites

The lab material assumes you have prior experience with:

- **Zephyr RTOS environment**
- **C language programming**

PC Setup

You can follow the instructions provided in the **Supplement A** document.

Microchip Development Tool Requirements

Name	Manufacturer	Part Number	
CABLE, USB A TO MICRO-B, 6 ft	Microchip	*CAB0028*	
SAM E54 Xplained Pro	Microchip	ATSAME54-XPRO	

Software Requirements

Name	Manufacturer	Version	
7-Zip	Igor Pavlov	v26.00+	
CMake	Kitware (Open Source)	v4.2.3+	
Device Tree Compiler (DTC)	David Gibson	v1.6.1+	
Git™	Software Freedom Conservancy (Open Source)	v2.53.0+	
Gperf	Free Software Foundation	v3.1+	
Ninja	Ninja Project (Open Source)	v1.13.2+	
OpenOCD	Dominic Rath	v0.12.0+	
Python™	Python Software Foundation (Open Source)	v3.12+	
Visual Studio Code	Microsoft (Open Source)	v1.110+	
Wget	Free Software Foundation	v1.21.4+	
Zephyr OS	Linux Foundation	v4.3.0	
Zephyr SDK	Linux Foundation	v0.17.4	

Lab 1 - Run the Blinky sample project on your custom board

Purpose

In this lab, you will learn how to create Zephyr support for a custom board and run the Zephyr Blinky example on it.

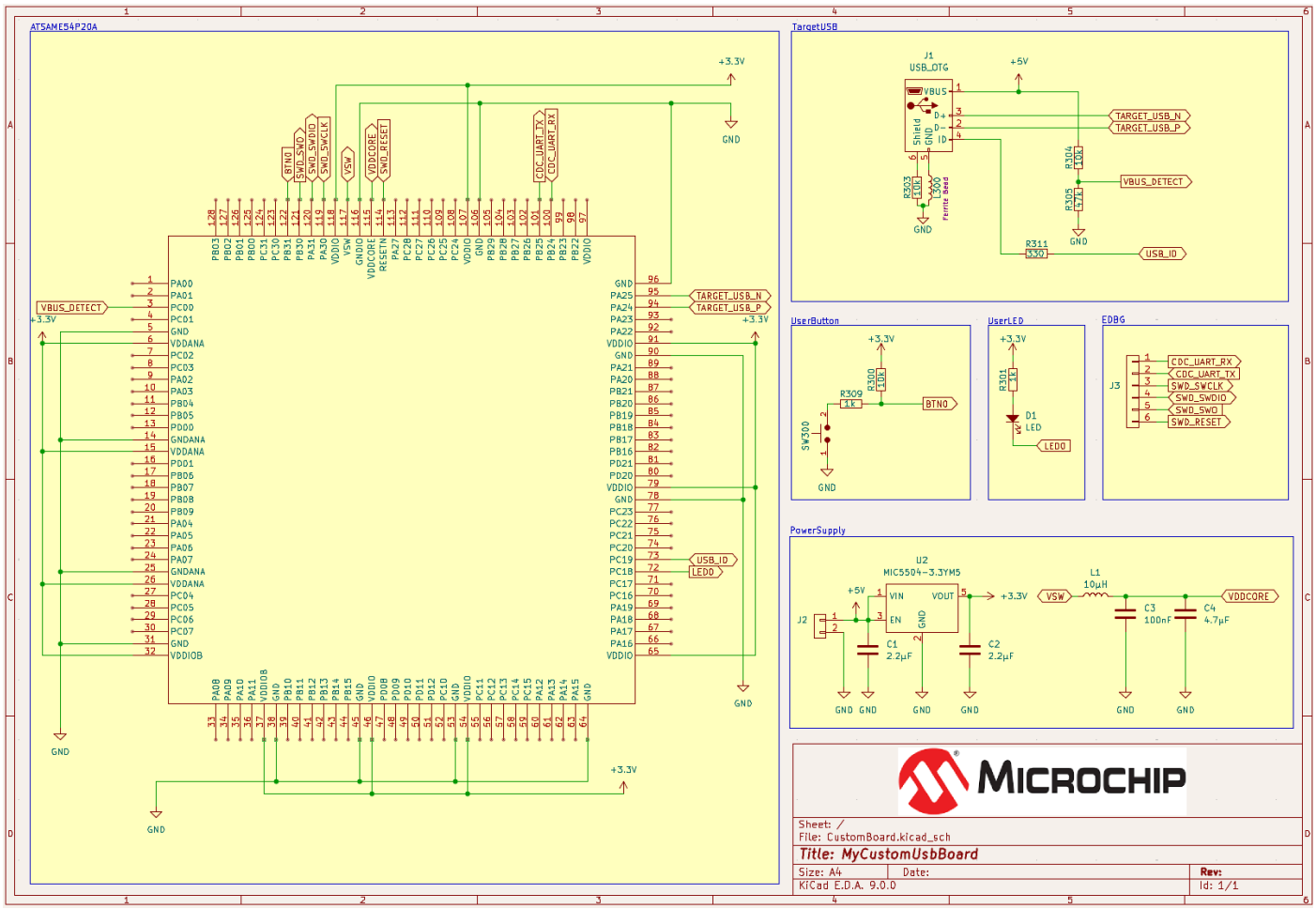
Overview

You'll create board support for a custom hardware platform by starting from a set of provided template files, defining the missing Devicetree and configuration files, then build and flash the Zephyr Blinky example. The application uses GPIO to control an LED and outputs messages over UART.

Upon successful completion, the LED on your board should blink and you should see serial output in a terminal, confirming that your Devicetree, UART configuration, and board support are correctly implemented.

Procedure

Your task is to create the board support for your custom board that has the following schematic:



From the schematic you can recognize the following Pin Mapping:

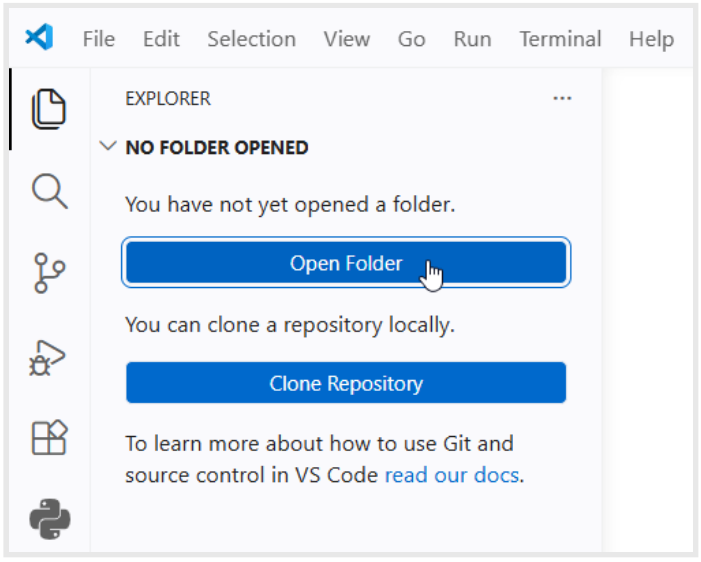
Function	Pin	Active Level	Notes
LED0	PC18	Active **LOW**	Lights up when pin = 0
Button SW0	PB31	Active **LOW**	Needs internal **pull-up**
UART TX	PB25	-	
UART RX	PB24	-	
USB DP	PA24	-	
USB DM	PA25	-	
Vbus	PC00		High when USB cable connected

The File Structure you need to create looks as follow:

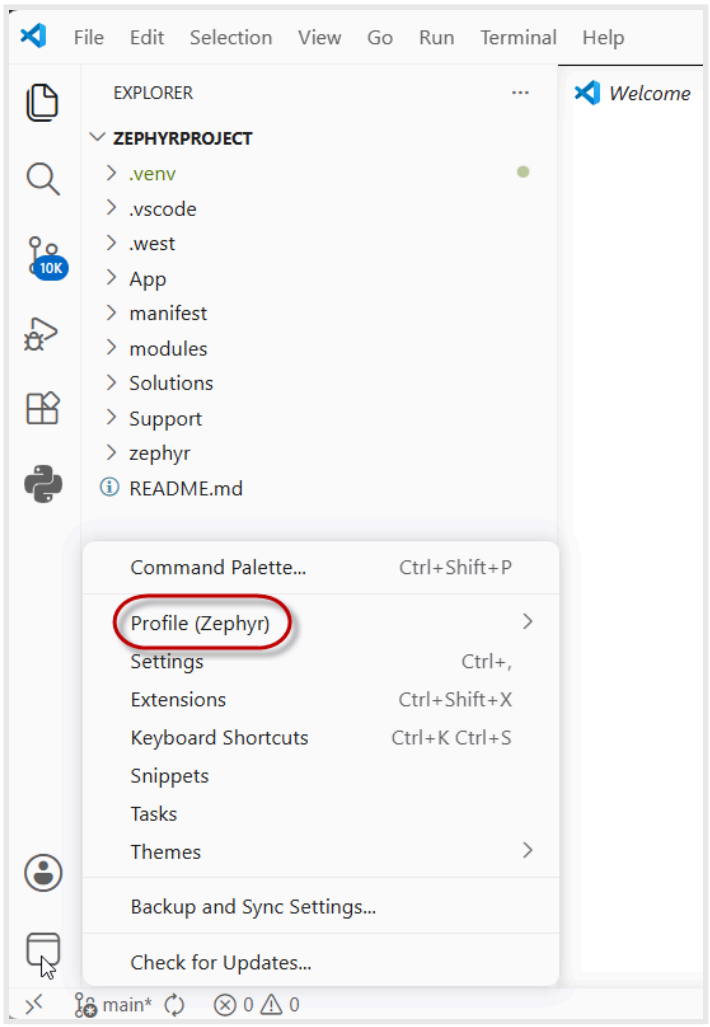
```
zephyrproject\                          ← your working folder
└── boards\
    └── my_board\
        ├── board.yml                   ⭐ TO CREATE from scratch
        ├── Kconfig.my_board            ⭐ TO CREATE from scratch
        ├── my_board.dts                🖍️  EDIT (has TODOs)
        ├── my_board_defconfig          🖍️  EDIT (has TODO)
        ├── board.cmake                 ✓  use as-is
        ├── pre_dt_board.cmake          ✓  use as-is
        └── support\
            └── openocd.cfg             ✓  use as-is
```

You will create from scratch the board metadata file and the kconfig file of your custom board, then you will copy the remaining files from the class Support folder. The files marked **[TODO]** contain code already and you will need to complete it. The other files are provided to be used as they are because they are very SoC-specific, or related to compile and debug processes, that depends on which tools you will use. You can create folders and files from **Visual Studio Code (VSC)**.

Launch **VSC** and open the **west workspace** folder (**C:/MASTERs/26021_RTOS5/zephyrproject**); if another folder is already open in VSC, close it and open the west workspace folder of this class.



Verify that you are using the Zephyr profile to enable all the **VSC Extensions** that you need to use for this class:



If a different profile is selected, you need to select the Zephyr profile.

First you need to create the **boards** folder in your **west workspace**, then you need to create a folder with the name you will use for your board. In this class you will use **my_board** as board name. Finally, you need to copy the files that you will modify in the following steps and the other ones you will use as they are from the **Support** folder into **my_board** folder.

You can copy the needed files from the **Support** folder in the **my_board** folder using the provided batch file:

> **C:\MASTERs\26021_RTOS5\Copy_Lab1_Files.bat**

or just run the following command from the VSC terminal:

```
..\Copy_Lab1_Files.bat
```

Copy

Note: the batch file also creates the needed folders so you don't need to create them before launching the batch file.

There are two files that you need to create from scratch:

- **board.yml**
- **Kconfig.my_board**

1 Create the **board.yml** file

Inside the **my_board** folder, create the **board.yml** file, where you need to define the YAML structure that describe the board.

> HINT:
>
> Add the YAML structure **board** with four fields:
>
> - **name:** must match the directory name (my_board)
> - **full_name:** human-readable description (My Board)
> - **vendor:** team identifier (my-company)
> - **socs → name:** link to the SoC (same54p20a)
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/latest/hardware/porting/board_porting.html

Step 1 - Solution

2 Create the **Kconfig.my_board** file

Inside the **my_board** folder, create the **Kconfig.my_board** file, where you register your board within the build system.

> HINT:
>
> Add two lines:
>
> - A config directive defining the board symbol.
>   Convention: BOARD_<UPPERCASE_BOARD_NAME>.
> - A select statement that auto-enables the matching SoC. Convention:
>   SOC_<UPPERCASE_SOC_NAME>
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/latest/hardware/porting/board_porting.html

Step 2 - Solution

Now in your **my_board** folder you have all the needed files to support your custom board. There are still two files that need to be completed.

3 Complete **my_board.dts** file

Open **my_board.dts** file and add the declaration for LED and Button nodes, then enable SERCOM 2 and USB nodes.

- **LED node:** replace ***/* add led0 here */*** inside the ***leds { ... }*** block with a child node describing the **Yellow LED**. Give it a node label, so the existing alias can resolve it, ***gpios*** and ***label*** properties to configure the node.
- **Button node:** replace ***/* add button0 here */*** inside the ***buttons { ... }*** block with a child node describing the user button **SW0**, including a node label for the existing alias. The following ***vbus0*** node is your template, but you need to also enable the pull-up resistor on that pin.

HINT:

LED node: replace /* add led0 here */ inside the leds { ... } block with a child node describing the **Yellow LED**. Give it a label so the existing alias ***led0 = &led0*** can resolve it.

Button node: replace /* add button0 here */ inside the buttons { ... } block with a child node describing the user button (SW0). The existing vbus0 node next to it is your template.

Remember:

- GPIO ports use labels (&porta, &portb, &portc, &portd) and port pin number followed by GPIO flags.
  Pin PC18 → ***<&portc 18 (FLAGS)>***.
- Multiple GPIO flags can be OR'd: ***(GPIO_PULL_UP | GPIO_ACTIVE_LOW)***.
- Valid **INPUT_KEY_*** constants are in **<west_workspace>\zephyr\include\zephyr\dt-bindings\input\input-event-codes.h**. Use ***INPUT_KEY_0*** for ***button0*** because ***vbus0*** already use ***INPUT_KEY_1***.

Enabling USB (labeled ***zephyr_udc0***) and ***&sercom2*** nodes requires you to change their ***status*** from ***disabled*** to ***okay***.

Documentation is available here:

https://docs.zephyrproject.org/latest/hardware/porting/board_porting.html

Step 3 - Solution

4 Complete **my_board_defconfig** file

Find the comment ***# add UART console, interrupt driven here*** and add the two missing CONFIG_*=y lines that:

- route the console output to UART
- enable the interrupt-driven UART driver

> HINT:
>
> You can search the correct **Kconfig** options
> here: **https://docs.zephyrproject.org/latest/kconfig.html**
>
> Search for "console" to find the correct one to ***Use UART for console***.
>
> Search for "uart interrupt" to enable ***UART Interrupt support***.
>
> Documentation is available here:
>
> **https://docs.zephyrproject.org/latest/hardware/porting/board_porting.html**

Step 4 - Solution

Now you completed the creation of your custom board Zephyr support.

You will use the Zephyr **Blinky sample** project to test the board support you created; you need to create a folder named Lab in the **C:\MASTERs\26021_RTOS5\zephyrproject** folder and copy there the sample project files that you find in **C:\MASTERs\26021_RTOS5\zephyrprojec\zephyr\samples\basic\blinky** folder.

> Not all the files that are in the **C:\MASTERs\26021_RTOS5\zephyrprojec\zephyr\samples\basic\blinky** are needed, only:
>
> - **CMakeList.txt**
> - **prj.conf**
> - **src\main.c**

> You can copy the needed files from the Blinky Sample project in the **Lab** folder using the provided batch file:
>
> **C:\MASTERs\26021_RTOS5\Copy_Blinky_Sample.bat**
>
> or just run the following command from the VSC terminal:

```
..\Copy_Blinky_Sample.bat
```

Copy

If you try to build the newly created **Lab** project just using **my_board** as board parameter you will get the error "**No board named 'my_board' found**" because you need to specify the folder containing your **boards** folder.

To build the project, you need to add, to the ***west build*** command, the parameter ***-- -DBOARD_ROOT=".."***, because your **boards** folder is in the parent folder of the **Lab** folder. Instead of adding that parameter every time you need to build the project, you can modify your project **CMakeLists.txt** file to have west finding your custom board automatically.

5 Edit **CMakeLists.txt** file

Append the list of folders contained in the **CMake** variable ***BOARD_ROOT***, adding the parent folder (west workspace)

> HINT:
>
> You can use ***list(APPEND ...)*** to add ***${CMAKE_CURRENT_SOURCE_DIR}/..*** to the variable ***BOARD_ROOT***
>
> Append the list before the line containing ***find_package(..)***
>
> Documentation is available here:
>
> **https://docs.zephyrproject.org/latest/develop/application/index.html#important-build-system-variables**

Step 5 - Solution

The schematic of your custom board has the same connections of the **SAME54 Xplained Pro** board so you can test the board support you created using the provided demo board.

Build and program

You can now build the application using **west** in the **VSC terminal**:

```
west build -b my_board -p always .\Lab\
```

Copy

Connect the "Debug USB" port of the **SAME54 Xplained Pro** to an available USB port of the PC, then deploy the executable using **west** again:

```
west flash
```

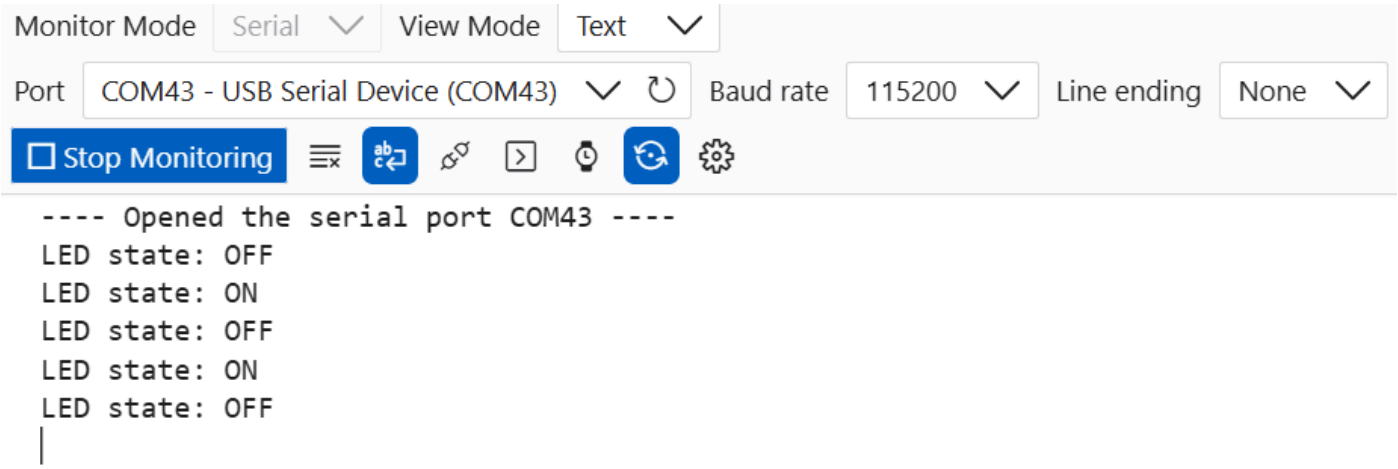
Copy

In the **Serial Monitor** Extension, open the **Port** corresponding to the **EDBG** serial bridge at **115200** baud rate.

Results

You should see two things:

1. **Yellow LED blinks** at 1 Hz on the board.
2. **Console output** on the EDBG virtual COM port.



Summary

Congratulations! You have successfully created the board support for your custom board, starting from your board schematic and using the already available support for the SOC that will execute the code.

Continue to Lab 2...

Lab 1: Step 1 - Solution

Copy & Paste

```yaml
board:
  name: my_board
  full_name: My Board
  vendor: my-company
  socs:
    - name: same54p20a
```

Copy

Go back to Lab 1

Lab 1: Step 2 - Solution

Copy & Paste

```
config BOARD_MY_BOARD
 select SOC_SAME54P20A
```

Copy

Go back to Lab 1

Lab 1: Step 3 - Solution

Copy & Paste

Replace the ***/* add led0 here */*** with this one:

```
led0: led_0 {
 gpios = <&portc 18 GPIO_ACTIVE_LOW>;
 label = "Yellow LED";
};
```

Copy

Replace the ***/* add button0 here */*** with this one:

```
button0: button_0 {
 gpios = <&portb 31 (GPIO_PULL_UP | GPIO_ACTIVE_LOW)>;
 label = "SW0";
 zephyr,code = <INPUT_KEY_0>;
};
```

Copy

Go back to Lab 1

Lab 1: Step 4 - Solution

Copy & Paste

```
CONFIG_UART_CONSOLE=y
CONFIG_UART_INTERRUPT_DRIVEN=y
```

Copy

Go back to Lab 1

Lab 1: Step 5 - Solution

Copy & Paste

```
list(APPEND BOARD_ROOT ${CMAKE_CURRENT_SOURCE_DIR}/..)
```

Copy

Go back to Lab 1

Lab 2 - Redirect the Console to the USB

Purpose

In the previous lab, you viewed the serial output of your device through the USB bridge provided by the onboard programmer/debugger (**EDBG**).

In this lab you will learn how to move the communication from **UART** (SERCOM1) to **USB**

Overview

After redirecting the serial communication to **USB**, you will use the **Console** service, provided by **Zephyr OS**, to toggle **LED0,** if you receive the **"T"** character from the **Serial Monitor** Extension of **Visual Studio Code**, otherwise you will send back a message with the echo of the received character; if **SW0** on the board is pressed, you will send the string **"Button pressed!\n"** to the **Serial Monitor** (via **Console** service).

Procedure

This lab will continue from the previous lab. If you haven't finished the previous lab, please overwrite the content of **board** and **Lab** folders with the **Solution/boards** and **Solution/lab1** folders.

You can copy the solution of Lab 1 over Lab and boards folders using the provided batch file:

> **C:\MASTERs\26021_RTOS5\Copy_Lab1_Solution.bat**

or just run the following command from the Visual Studio Code terminal:

```
..\Copy_Lab1_Solution.bat
```

Copy

Instead of modifying the custom board Devicetree, you can use the Devicetree Overlay in your application.

1 Add a Devicetree Overlay

Create a new ***app.overlay*** file in the ***Lab*** folder. Redirect the Console from **SERCOM2** to **USB** and configure the USB node to work as **USB CDC-ACM** Device.

HINT:

Overlay***hosen*** node is ***zephyr, console*** and you need to set it to ***&cdc_acm_uart***

You can use the ***zephyr_udc0*** node label to overlay the configuration of USB peripheral adding a child node labeled ***cdc_acm_uart*** with ***compatible*** property set to "***zephyr, cdc-acm-uart***"

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/build/dts/intro-syntax-structure.html

https://docs.zephyrproject.org/latest/build/dts/api/bindings/serial/zephyr%2Ccdc-acm-uart.html

Step 1 - Solution

2 Enable software support for the **USB Device Stack**, the **USB CDC-ACM** Class, the **Console** and the **Input** services

You can do that by adding the proper Kconfig options to the ***prj.conf*** file.

HINT:

The **Kconfig options** that you need to add are:

- **CONFIG_USB_DEVICE_STACK_NEXT:** adds the USB Device Stack support
- **CONFIG_USBD_CDC_ACM_CLASS:** adds the USB CDC-ACM Class support
- **CONFIG_CDC_ACM_SERIAL_INITIALIZE_AT_BOOT:** initialize the stack at boot
- **CONFIG_CDC_ACM_SERIAL_ENABLE_AT_BOOT:** enable the stack at boot
- **CONFIG_CDC_ACM_SERIAL_VID:** assign the Vendor ID
- **CONFIG_CDC_ACM_SERIAL_PID:** assign the Product ID
- **CONFIG_CDC_ACM_SERIAL_MAX_POWER:** assign the max current drained by the device
  (Value = max current / 2mA). For a self-powered USB device, it should be 100mA.
- **CONFIG_CDC_ACM_SERIAL_MANUFACTURER_STRING:** assign the Manufacturer String
- **CONFIG_CDC_ACM_SERIAL_PRODUCT_STRING:** assign the Product String
- **CONFIG_CONSOLE_SUBSYS:** adds Console service support
- **CONFIG_CONSOLE_GETCHAR:** enable character by character input and output
- **CONFIG_INPUT:** add Input service support

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/kconfig.html

Step 2 - Solution

3 Modify the example code

Open ***main.c*** file and locate the ***#include*** directives at the beginning of the file; unlike the Blinky demo, you don't need to include the header file of ***stdio***, but you need to include the header files of ***Console*** (zephyr/console/console.h) and ***Input*** (zephyr/input/input.h).
In this second lab, you are going to use the reference ***button0*** (**SW0**).

HINT:

For the button0 (SW0), you need to get the code property of the button and you can define it as SW0_CODE using the DT_PROP macro and the DT_NODELABEL macro to refer to SW0 (button0)

Documentation is available here:

https://docs.zephyrproject.org/latest/build/dts/api/api.html#node-identifiers-and-helpers

Step 3 - Solution

4 Define the **Input** Callback Function

You are using the Input service that receive events from the **gpio-keys driver** and dispatch them. You can define a callback function, that prints the desired string when you press the **SW0** button, and register that function in the **Input** subsystem.

HINT:

You can define a function named ***input_cb()*** that you will register in the **Input** service using the **INPUT_CALLBACK_DEFINE** macro. You can use **NULL** for ***_dev*** and ***_user_data*** parameters because they are not used in this lab. Define the function immediately before the ***main()*** function.

The function needs to have a prototype like the following:
void input_cb(struct input_event *event, void *user_data);

In the ***input_cb*** function, you need to check for the ***event->code*** and if it is related to **SW0** button (***SW0_CODE***) then you need to check if the event value is pressed (***event->value == true***); if both the condition are met, you can use the function ***printk()*** to print the ***"Button pressed!\n"*** string.

Documentation is available here:

https://docs.zephyrproject.org/latest/services/input/index.html

Step 4 - Solution

5 Initialize the **Console**

When you press a character key on the PC keyboard, you need to receive it and, based on the received character, you need to toggle **LED0** or print a message with the echo of the received character.
To receive a character, you can use the **Console** service that needs to be initialized. You can use the ***console_init()*** function to initialize the **Console** service.

HINT:

You need to call ***console_init()*** function before invoking other **Console** APIs. You can call it in the ***main()*** function, before the ***while(1)*** loop.

console_init() function returns 0 on success or an error code (<0) otherwise. It is a good practice to check the returned value and do not continue if there is an error (***return 0***)

You can now print a string to let the user know what the application does (e.g. ***printk("\nPress T on the keyboard to toggle the LED 0\n - or - \nPress SW0 on the board to receive a message\n")***)

Documentation is available here:
https://docs.zephyrproject.org/latest/doxygen/html/group__console__api.html

Step 5 - Solution

Inside the ***while(1)*** loop of the ***main()*** function, there is the code of the Blinky example.

You can reuse it, commenting-out the ***k_msleep()*** function call because in this lab we do not need the 1 second delay anymore; the remaining code inside the loop is what you need to do if the received character is "***T***".

6 Toggle **LED0** when character "T" is received

Now modify the code to receive a character, using the Console API, and toggle **LED0**, if you received the character "***T***", or send back a string stating which character has been received.

> HINT:
>
> You can use ***console_getchar()*** function to receive a character. You can assign the returned character in an unsigned char variable (***chr***), then test the value of that variable: if it is "***T***" you can use the Blinky code to toggle **LED0** and print the LED state (remember to comment-out the delay to avoid low responsiveness), otherwise you need to send back a string echoing the received character using ***printk("Received character: %c\n", chr)***
>
> Documentation is available here:
> **https://docs.zephyrproject.org/latest/doxygen/html/group__console__api.html**

Step 6 - Solution

Build and program

You can now build the application using **west** in the **VS Code terminal**:

```
west build -b my_board -p always Lab
```

Copy

Then you can deploy the executable using **west** again:

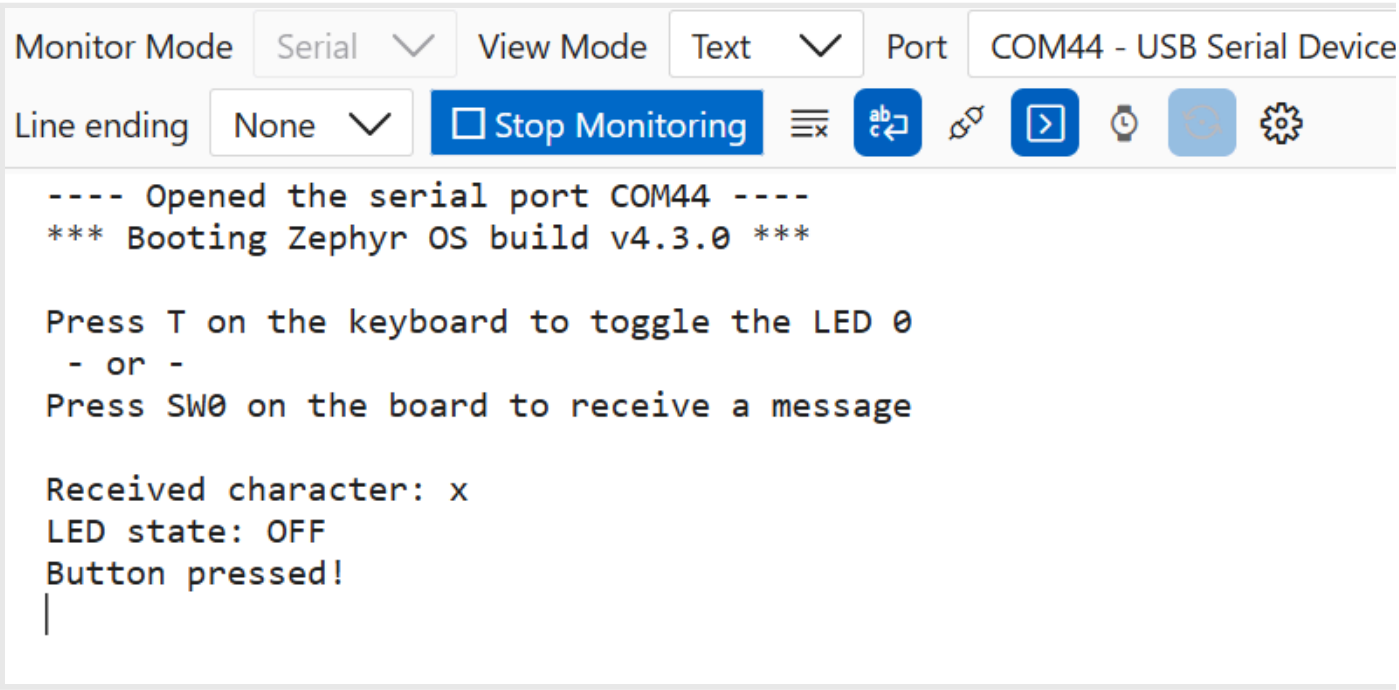
```
west flash
```

Copy

Connect the second USB cable, connected to "**TARGET USB**" port on the board, to an available USB port of the PC and use the **Serial Monitor** Extension, opening the **Port** corresponding to the **USB CDC-ACM** Device at **115200** Baude rate.

Results

If you press "**x**", then "**T**" and then the button **SW0**, you should see something like the following screenshot:



Summary

Congratulations! You have successfully redirected the **Console** to **USB** and communicated with the **Serial Monitor** running on the PC!

This lab demonstrated how it is easy to "just" add USB communication, redirecting the **Console** service.

Extra Question

Have you noticed a strange behavior? The string you are sending after enabling the **Console** is not displayed in the **Serial Monitor** window until you press a key or the **SW0** button on the board.

Can you guess why?

You can use **MPLAB AI Coding Assistant** to search for a possible reason...

Continue to Lab 3...

Lab 2: Step 1 - Solution

Copy & Paste

Copy

```
/ {
    chosen {
        zephyr,console = &cdc_acm_uart;
    };
};

&zephyr_udc0 {
    cdc_acm_uart: cdc_acm_uart {
        compatible = "zephyr,cdc-acm-uart";
        label = "Zephyr USB CDC-ACM";
    };
};
```

Go back to Lab 2

Lab 2: Step 2 - Solution

Copy & Paste

Copy

```
CONFIG_INPUT=y
CONFIG_USB_DEVICE_STACK_NEXT=y
CONFIG_USBD_CDC_ACM_CLASS=y
CONFIG_CDC_ACM_SERIAL_INITIALIZE_AT_BOOT=y
CONFIG_CDC_ACM_SERIAL_ENABLE_AT_BOOT=y
CONFIG_CDC_ACM_SERIAL_VID=0x04D8
CONFIG_CDC_ACM_SERIAL_PID=0x000A
CONFIG_CDC_ACM_SERIAL_MAX_POWER=50
CONFIG_CDC_ACM_SERIAL_MANUFACTURER_STRING="Microchip Technology Inc."
CONFIG_CDC_ACM_SERIAL_PRODUCT_STRING="Zephyr CDC-ACM Console"
CONFIG_CONSOLE_SUBSYS=y
CONFIG_CONSOLE_GETCHAR=y
```

Go back to Lab 2

Lab 2: Step 3 - Solution

Copy & Paste

Copy

```
#include <zephyr/console/console.h>
#include <zephyr/input/input.h>

#define SW0_CODE DT_PROP(DT_NODELABEL(button0), zephyr_code)
```

Go back to Lab 2

Lab 2: Step 4 - Solution

Copy & Paste

Copy

```
void input_cb(struct input_event *event, void *user_data)
{
    if (event->code == SW0_CODE) {
        if (event->value) {
            printk("Button pressed!\n");
        }
    }
    return;
}
INPUT_CALLBACK_DEFINE(NULL, input_cb, NULL);
```

Go back to Lab 2

Lab 2: Step 5 - Solution

Copy & Paste

Copy

```
    ret = console_init();
    if (ret < 0)
    {
        return 0;
    }
    printk("\nPress T on the keyboard to toggle the LED 0\n - or -\nPress SW0 on the board to receive a
message\n\n");
```

Go back to Lab 2

Lab 2: Step 6 - Solution

Copy & Paste

Replace the ***while(1)*** loop inside the ***main()*** function with this one:

Copy

```
    while (1) {
        unsigned char chr = console_getchar();
        if (chr == 'T' )
        {
            ret = gpio_pin_toggle_dt(&led);
            if (ret < 0) {
               return 0;
            }

            led_state = !led_state;
            printf("LED state: %s\n", led_state ? "ON" : "OFF");
            // k_msleep(SLEEP_TIME_MS);
        }
        else
        {
            printk("Received character: %c\n", chr);
        }
    }
```

Go back to Lab 2

Lab 3 - Build a CDC device and communicate with a terminal on PC

Purpose

In this lab you will learn how to configure the **USB Device Stack** to implement an **USB CDC-ACM** Device

Overview

Instead of just redirecting the **Console**, you will configure the **USB Device Stack** from scratch, creating the USB Device Controller context structure, defining the USB Device strings descriptors, the USB Configuration descriptors, the USB message notification callback function and, finally, you will initialize and enable the USB Device Stack.

Once enabled, wait for the Line Control signal, that indicate the PC terminal opened the communication with the USB device, before sending the initial string.

You will use the **interrupt-driven API** to send and receive data, like you did in the previous lab.

Procedure

This lab will continue from the previous lab. If you haven't finished the previous lab, please overwrite the content of **Lab** folder with the content of **Solution/lab2** folder.

You can copy the solution of Lab 2 over Lab folder using the provided batch file:

> **C:\MASTERs\26021_RTOS5\Copy_Lab2_Solution.bat**

or just run the following command from the Visual Studio Code terminal:

```
..\Copy_Lab2_Solution.bat
```

Copy

First, you need to remove the redirection of the **Console** from the USB peripheral, removing the ***/chosen*** node you added in the previous lab from the **app.overlay** file.

You are going to configure the **USB Device Stack** in your code so you need to remove the ***CDC_ACM_SERIAL*** configuration options from the **prj.config** file and remove the **Console** configuration options as well, because you will not use the **Console API** in this lab. Finally, you need to add the configuration options for the **Serial** and **Hardware Information** drivers to use the **UART API** for the **Virtual CDC ACM UART** and enable the Serial Number string for the **USB CDC_ACM Device**.

1 Remove **Console** and add **Serial** support

You can do that by modifying the ***prj.conf*** file.

> HINT:
>
> The **Kconfig options** that you need to add are:
>
> - **CONFIG_SERIAL:** enables the Serial drivers support
> - **CONFIG_UART_INTERRUPT_DRIVEN:** adds interrupt support for **UART**
> - **CONFIG_UART_LINE_CTRL:** enables the API to control the serial line
> - **CONFIG_HWINFO**: enable the **Hardware Information driver**
>
> Documentation is available here:
>
> **https://docs.zephyrproject.org/4.3.0/kconfig.html**

Step 1 - Solution

To improve the code readability , you can add the **USB Device Stack** configuration in separate files. Create two files, **usb.c** and **usb.h,** in the **Lab/src** folder.

In this lab you will use the **USB Device Controller APIs** to implement a Virtual UART so, in **usb.h** file, you need to include the header file of the USB module (zephyr/usb/usbd.h) and the one for the UART (zephyr/drivers/uart.h) because you will use UART API to send and receive data through the Virtual UART; then, including **usb.h** in the **usb.c** file, you can call the provided APIs.

2 Define the **USB CDC-ACM** device structure

You need the device structure related to the **USB CDC-ACM Device** instance to interact with it using the **UART Interrupt-driven APIs**.

HINT:

You can use the **DEVICE_DT_GET** macro to get a reference to the CDC interface that has ***cdc_acm_uart*** label in the **app.overlay** file; **DEVICE_DT_GET** requires a node identifier that you can retrive from the label using **DT_NODELABEL** macro.

In **usb.c** file, define a device constant structure named ***cdc_dev*** and assign to it the reference to the ***cdc_acm_uart*** interface.

In **usb.h** file, declare ***cdc_dev*** as external, so you will be able to use it in the **main.c** file

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group__device__model.html

https://docs.zephyrproject.org/latest/doxygen/html/group__devicetree-generic-id.html

Step 2 - Solution

The USB Device Stack requires a context structure that you can define in the **usb.c** file.

3 Define the **USB Device Controller context structure**

You can use the **USBD_DEVICE_DEFINE** macro to create a context structure to initialize the USB Device stack.

HINT:

Use the marco to create the structure named ***usbd_ctx***, assigning **0x04D8** as **Vendor ID** and **0x000A** as **Product ID**; you can get a reference to the device structure of the **USB Device Controller**, labeled ***zephyr_udc0,*** using the two macros **DEVICE_DT_GET** and **DT_NODELABEL** like you have done in the previous step.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group__devicetree-generic-id.html

https://docs.zephyrproject.org/latest/doxygen/html/group__usbd__api.html

Step 3 - Solution

4 Define the **USB Device String Descriptors**

You can use USB Device Stack macros to define the USB Device String Descriptors.

HINT:

You can use the following macros:

- **USBD_DESC_LANG_DEFINE:** defines a descriptors node used as language string descriptor zero.
  Use it to define the ***lang_desc*** string descriptor
- **USBD_DESC_MANUFACTURER_DEFINE:** defines a descriptors node used as manufacturer string descriptor.
  Use it to define the ***mfr_desc*** string descriptor initialized with ***"Microchip Technology Inc."***
- **USBD_DESC_PRODUCT_DEFINE**: defines a descriptors node used as product string descriptor.
  Use it to define the ***prod_desc*** string descriptor initialized with ***"Zephyr CDC-ACM"***
- **USBD_DESC_SERIAL_NUMBER_DEFINE**: defines a descriptors node used as serial number string descriptor.
  Use it to define the ***sn_desc*** string descriptor (it requires that **CONFIG_HWINFO** is enabled in **prj.conf** file).

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group__usbd__api.html

Step 4 - Solution

5 Define the **USB Device Configuration**

The configuration descriptor and the USB Configuration instance can be created using the provided macros.

HINT:

Use the macro **USBD_DESC_CONFIG_DEFINE** to create the ***fs_cfg_desc*** configuration descriptor, initialized with "***FS Configuration***", and the macro **USBD_CONFIGURATION_DEFINE** to create the ***fs_config*** configuration instance.

The second macro requires to declare if the device is a self or bus powered USB Device and the maximum current the device will drain from the USB Host: the value is a number that return the current value in 2mA steps (250 * 2mA = 500mA). Declare your device as bus powered USB device (**USB_SCD_SELF_POWERED**) that drain 100mA.

Please note that the ***attrib*** parameter of the second macro has defines only for setting the flags: ***attrib*** will be 0 for a Bus Powered USB device that does not implement Remote Wakeup (both flags set to zero)

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group__usbd__api.html

Step 5 - Solution

You should manage the USB Message notifications and you can define a callback function to handle them and register it into the USB Device Stack.
For example, you need to enable or disable the USB Device support based on message related to Vbus pin: if it is VBUS_READY (cable connected) you need to enable USB Device support, otherwise, if it is VBUS_REMOVED (cable disconnected), you need to disable it.

6 Define the **USB Message notifications** Callback function

You will use this callback to manage the **USB Message notification** relevant for your application. In this lab you only handle the messages related to a **UDC driver** that support **VBUS** detection. If the **UDC driver** does not provide that support, you need to use a **GPIO** to check when the cable is connected. Writing the code to handle both cases, based on the capability of the **UDC driver**, makes your code more portable.

HINT:

You can define a callback function and register it with the registration function provided by the **USB Device Core API**.

The prototype of the callback function is ***static void usb_msg_callback(struct usbd_context *const ctx, const struct usbd_msg *const msg);***

Define the callback function and enable or disable the **USB Device Stack** when the **USB Device Controller driver** can detect **VBUS** state changes: enable when the ***type*** member of the ***msg*** parameter is **USBD_MSG_VBUS_READY**, disable it when it is equal to **USBD_MSG_VBUS_REMOVED**

Like in the previous lab, you need to send an initial string explaining what the application does when connected so you will define, in a following step, a function that does that, with the following prototype:

- ***void manage_usb_device(bool connected)***

You can call that function to enable (passing ***true***) or disable (passing ***false***) the USB Device support.

You can check the **VBUS** Detection capability of the **USB Device Controller** driver with the function:

- ***int usbd_can_detect_vbus(struct usbd_context *const uds_ctx)***

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html

Step 6 - Solution

Now you need to add USB String Descriptors, Configuration Descriptor and register the callback function in the USB Device Stack context structure. You can do that in a function that you can define in **usb.c** file and call it from **main.c** file. As mentioned in the previous step, you need handle also the case where the **UDC driver** does not detect Vbus, so you need to monitoring it in your code.

7 Define the **Vbus flag** and the **USB initialization function**

In **usb.c** file, define a function named ***init_usb_device()***, without parameter, that returns an integer. Define a boolean variable, named ***managed_vbus***, to store the information about the Vbus detection capability of the **UDC driver**.
In **usb.h** file, declare the function prototype and the boolean variable as external to be used in the **main.c** file.

> HINT:
>
> The **USB Device Core APIs** you will call to initialize the stack return an error value (0 == no error)
>
> In case of error, the ***init_usb_device()*** function will return the error code to the caller function.
>
> Declare an integer variable ***err***, initialized at ***0***, at the beginning of the function, and return it at the end of the function.

Step 7 - Solution

8 Add **String Descriptors**

You need to add the previously defined USB Device String Descriptors.

> HINT:
>
> You can use the function ***int usbd_add_descriptor(struct usbd_context *const uds_ctx, struct usbd_desc_node *const desc_nd)*** to add the strings to the ***usbd_ctx*** structure:
>
> - ***lang_desc***
> - ***mfr_desc***
> - ***prod_desc***
> - ***sn_desc***
>
> Remember to assign the returned value to ***err*** and return it in case of error.
>
> Documentation is available here:
>
> **https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html**

Step 8 - Solution

9 Add **Configuration Descriptor**

You need to add the configuration descriptor to the UDC Context Structure.

> HINT:
>
> You can add the configuration ***fs_config*** to ***usbd_ctx*** using ***int usbd_add_configuration(struct usbd_context *const uds_ctx, const enum usbd_speed speed, struct usbd_config_node *const cfg_nd)*** function. The second parameter is the USB Speed and the **ATSAME54P20A** only support Full-Speed (**USBD_SPEED_FS**).
>
> Remember to assign the returned value to ***err*** and return it in case of error.
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html

Step 9 - Solution

10 Register the **USB CDC-ACM** Class

You need to register the USB Device Class in the context structure.

HINT:

You can register the **USB CDC-ACM** instance in ***usbd_ctx*** using ***int usbd_register_class(struct usbd_context *const uds_ctx, const char *name, const enum usbd_speed speed, const uint8_t cfg)*** function.

The second parameter is a unique identification string used to link a specific USB class instance; the standard identification strings used in Zephyr are:

- ***hid_n***: Used for Human Interface Devices
- ***cdc_acm_n***: Used for virtual UART controllers
- ***msc_n***: Used for Mass Storage Class instances
- ***uac2_n***: Used for USB Audio Class 2 devices
- ***cdc_ecm_n***: Used for Ethernet device emulation

where ***n*** is the instance number.

The third parameter is the USB Speed (**USBD_SPEED_FS**) and the forth is the configuration number (***1***).

Remember to assign the returned value to ***err*** and return it in case of error.

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html

Step 10 - Solution

11 Register the **USB Message notification** callback function

You need to register the USB Message notification callback function in the context structure.

> HINT:
>
> You can register the ***usb_msg_callback()*** function in ***usbd_ctx*** using ***int usbd_msg_register_cb(struct usbd_context *const uds_ctx, const usbd_msg_cb_t cb)*** function.
>
> Remember to assign the returned value to ***err*** and return it in case of error.
>
> Documentation is available here:
>
> **https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html**

Step 11 - Solution

12 Initialize the **USB Device Stack**

Now the USB Device Stack configuration for an USB CDC-ACM Device is completed and the stack can be initialized.

> HINT:
>
> You can Initialize the stack using the ***int usbd_init(struct usbd_context *const uds_ctx)*** function and the configured ***usbd_ctx*** context.
>
> Remember to assign the returned value to ***err*** and return it in case of error.
>
> Documentation is available here:
>
> **https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html**

Step 12 - Solution

13 Assign the VBUS detect capability to the ***managed_vbus*** flag

You need to set the value of ***managed_vbus*** variable based on the capabilities of the **UDC Driver**.

> HINT:
>
> Like you did in Step 6, you can check the **VBUS** Detection capability of the **USB Device Controller** driver with the function:
>
> - ***int usbd_can_detect_vbus(struct usbd_context *const uds_ctx)***
>
> and assign the returned value to ***managed_vbus***.
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html

Step 13 - Solution

In interrupt-driven mode, you need to fill in the **Virtual UART FIFO** and enable the transmit Interrupt, to send something to the host, while you get a receive interrupt when the host send you data that you need to extract from the **Virtual UART FIFO**. Remember that in an USB communication, only the host can initiate a transfer: sending something to the host means prepare the data and wait that the host asks for them.

You are going to use two data structures to store received data and data to send.

14 Define the data buffers type

In **usb.h**, define a structure type, named ***vcp_buffer***, with two members: ***buf*** and ***lenght***. The type of ***length*** should be an unsigned 16-bit integer while ***buf*** should be an array of unsigned 8-bit integer; the size of the array should be enough to contain the larger amount of data you are expecting to receive/transmit.
You are going to send the initial message when the Serial Monitor opens the Virtual COM Port (VCP) associated with your board so declare an external variable named ***data_to_send*** of type ***vcp_buffer***.

> The typical size of buffers, in an USB application, is a multiple of the Endpoint size. In this lab you need to send a string of 99 characters and the Bulk Endpoint size for a Full-Speed USB Device is 64 byte so you can use 128 as size for both buffers

Step 14 - Solution

As already mentioned in **Step 6**, you are going to use a function to enable or disable the USB Device Stack.

15 Declare the ***manage_usb_device()*** function

In **usb.c** file, define a void function named ***manage_usb_device()***, with a boolean parameter named ***connected***.
In **usb.h** file, declare the function prototype to be called from the **main.c** file.

Step 15 - Solution

16 Define the *manage_usb_device()* function

Inside the ***manage_usb_device()*** function, you need to check if the cable is connected, relying on the ***connected*** parameter, and enable or disable the USB Device support accordingly. You also need to correct the behavior of the previous lab, waiting the Serial Monitor to open the communication with the board, before sending the first message to it.

HINT:

If ***connected*** is ***true***, it means cable is connected so you can enable the USB Device support with the function ***int usbd_enable(struct usbd_context *const uds_ctx)*** passing the pointer to the context structure you defined (***usbd_ctx***).

Then you can wait the DTR signal defining a variable of type ***uint32_t*** named ***dtr*** where to store the line control value and initialize it to ***0***; in a while loop, where you looping if ***dtr*** is still ***0***, you can call the ***uart_line_ctrl_get()*** function to check for Data Terminal Ready (**UART_LINE_CTRL_DTR**) and wait for 100ms using ***k_sleep()*** function in combination with the **K_MSEC** macro. After the while loop is ended, you can use ***sprintf()*** function to insert the initial strings, you were sending with ***printk()*** in the previous lab, in ***data_to_send.buf*** and assign the returned value to ***data_to_send.length***.

Finally, you enable TX interrupt with ***void uart_irq_tx_enable(const struct device * dev)*** function, where the ***dev*** parameter is the **Virtual UART** device instance (***cdc_dev***)

If ***connected*** is ***false***, it means cable is disconnected so you can disable the USB Device support with the function ***int usbd_disable(struct usbd_context *const uds_ctx)*** passing the pointer to the context structure you defined (***usbd_ctx***).

For simplicity, ***manage_usb_device()*** is a void function so you do not need to check the return values of the APIs for errors.

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html

https://docs.zephyrproject.org/latest/doxygen/html/group__uart__interrupt.html

https://docs.zephyrproject.org/latest/doxygen/html/group__thread__apis.html

https://docs.zephyrproject.org/latest/doxygen/html/group__clock__apis.html

Step 16 - Solution

You have completed the sequence to initialize the USB Stack that you will actually enable calling the ***init_usb_device()*** function from your application code. In this lab, you are not going to use the **Console** anymore so the ***printf()*** and ***printk()*** functions need to be commented out because they are not going to be redirected to the **Virtual UART**. Moreover, you will use interrupt driven approach so the content of the ***while(1)*** loop in your ***main()*** function will be moved into the callback function that handle Interrupt request of the **Virtual UART**.

in **main.c**, comment-out the include directive for the **Console** (zephyr/console/console.h), the call to ***printk()*** in the ***input_cb()*** callback function and, in the ***main()*** function, the calls to ***console_init(), printk()*** and the content of the ***while(1)*** loop. Comment-out also the declaration of the local variable ***led_state*** in the ***main()*** function to avoid the unused variable warning.

For simplicity, in this class we are using a single application thread and, having commented-out the blocking call to ***console_getchar()*** from the ***while(1)*** loop, you need to re-enable the call to ***k_msleep()*** function to allow the **Input service** to work properly.

Remove the comment from ***k_msleep(SLEEP_TIME_MS)*** in the ***while(1)*** loop to re-enable it.

Now you have removed the **Console** so you need to add the code to use the **Virtual UART**. To call **UART API** and the functions you defined in previous steps, you need to include the correct header files.

17 Include the Header file

Locate the last include directive in **main.c** file and include the header file you created in previous steps.

Step 17 - Solution

In **Step 14** you defined the structure type of the buffer you will use to send and receive data and declared the structure ***data_to_send*** as external to be used in bot **usb.c** and **main.c** files.

You need to define the two data structures before using them.

18 Define data structures

Before ***input_cb()*** callback function, define two data structures, ***received_data*** and ***data_to_send***, of type ***vcp_buffer***

Step 18 - Solution

The strings you were sending with ***printk()*** function inside the ***input_cb()*** function, now require you to place them into ***data_to_send*** buffer and enable TX interrupt to be sent.

19 Prepare data to be sent ("**Button pressed!**\n\r")

Prepare the string that need to be sent when the button is pressed.

> HINT:
>
> You can use ***sprintf()*** function to insert the string, you were sending with ***printk()*** previously, in ***data_to_send.buf*** and assign the returned value to ***data_to_send.length***.
>
> You can then enable TX interrupt with ***void uart_irq_tx_enable(const struct device * dev)*** function, where the dev parameter is the **Virtual UART** device instance (***cdc_dev***)
>
> Documentation is available here:
>
> **https://docs.zephyrproject.org/latest/doxygen/html/group__uart__interrupt.html**

Step 19 - Solution

You need to handle the TX and RX IRQs related to the Virtual UART (USB peripheral) and you can register a callback function that it is called when there is an event (interrupt) related to it.

20 Define the **Virtual UART Callback** function

In the callback function, while there is a pending interrupt, you need to handle the event.

HINT:

Define ***uart_irq_cb()*** callback function to handle the events, just before the ***main()*** function. The prototype of that function needs to be:
void uart_irq_cb(const struct device *dev, void *user_data);

Define , inside the function, a static boolean variable (***led_state***) that you will use to know the status of **LED0**, initializing it to true.

Add a ***while*** loop that run until the irq update function (***int uart_irq_update(const struct device * dev)***), that start processing interrupts in ISR, and irq pending function

(***int uart_irq_is_pending(const struct device * dev)***) are both true.

You will fill the content of the ***while*** loop in the following steps.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group__uart__interrupt.html

Step 20 - Solution

21 Manage the **RX** interrupt

Inside the while loop, you need to check if data is ready and, if available, extract it from the Virtual UART FIFO. Once extracted, you need to check the size of the received data (***>0***) and the content of the first byte: if it is the character '***T***', you need to toggle the LED, update the ***led_state*** value and print the status of the LED; otherwise you need to send a string with the echo of the received character. Remember to enable the TX interrupt.

HINT:

You can use the ***uart_irq_rx_ready()*** function to check if something has arrived; then you can use ***uart_fifo_read()*** function to extract the data from the FIFO and copy them into ***received_data.buf***, updating the size of received data, assign it to ***received_data.length***.
If length is higher than zero, you need to check the content of ***received_data.buf[0]*** and if it is '**T**', you can toggle the **LED** using ***gpio_pin_toggle_dt()*** function, then invert the ***led_state*** value, prepare a string, with ***sprintf()*** function, to inform the host on the status of the **LED**; if it is not the character '**T**', prepare a string to echoing the received character. After preparing the strings, you need to enable the TX interrupt.

You can reuse the code you commented-out in the while(1) of the main function as reference.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group__uart__interrupt.html

https://docs.zephyrproject.org/latest/doxygen/html/group__gpio__interface.html

Step 21 - Solution

22 Manage the **TX** interrupt

You also need to check if the device is ready to send something and fill-in the FIFO, if ready. After filling it, you need to disable the TX interrupt because the event TX complete is not implemented yet and, if you do not disable the interrupt, at soon as the FIFO is sent to the host, the device is ready and the same interrupt is triggered, refilling the FIFO with te same string, sending it again and again.

HINT:

You can use the ***uart_irq_tx_ready()*** function to check if the Virtual UART is ready to accept additional data to be sent to the host, then you can use ***uart_fifo_fill()*** function to copy the data from the buffer ***data_to_send.buf*** to the FIFO; you can use ***data_to_send.length*** as size parameter.

Finally you need to disable the TX interrupt with ***uart_irq_tx_disable()*** function.

uart_irq_tx_ready() function returns the number of bytes you can add to the FIFO while ***uart_fifo_fill()*** function returns the actual number of bytes that were added to the FIFO, from the buffer.
In this lab we are not checking the return value of ***uart_fifo_fill()***, but in your actual application it is a good practice to doublecheck if the number of bytes written is equal to the number of bytes you wanted.

Documentation is available here:

https://docs.zephyrproject.org/latest/doxygen/html/group__uart__interrupt.html

Step 22 - Solution

In **Step 19**, you removed the ***printk()*** call and substituted it with a call to ***sprintf()*** function, followed by the TX interrupt enable function to start the transmission. Actually, no communication will take place until you initialize the USB Device Stack and register the Virtual UART callback function.

In the ***main()*** function, go just before the ***while(1)*** loop and add the code related to the following steps immediately before it.

23 Initialize the **USB Device Stack**

First, you need to check if the device is ready for use and the device pointer is a valid handle, then you can call the initialization routine you defined in **usb.c** file.

> HINT:
>
> You can use the ***device_is_ready()*** function to verify if the device pointed by ***cdc_dev*** is ready; if not, you can return ***0*** to the kernel (ending the main thread).
>
> If the handle is valid, you can call the ***init_usb_device()*** function you previously defined and check the returned value: if it is ***<0*** then return ***0*** to the kernel.
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/latest/doxygen/html/group__device__model.htm

Step 23 - Solution

24 Register the **Virtual UART Callback** function

you need to register the ***uart_irq_cb()*** function and enable the RX interrupt.

> HINT:
>
> You can use the ***uart_irq_callback_set()*** function to register ***uart_irq_cb()*** for the Virtual UART associated with ***cdc_dev*** handle, and ***uart_irq_rx_enable()*** to enable its RX interrupt.
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/latest/doxygen/html/group__uart__interrupt.html

Step 24 - Solution

In **Step 6** you defined the USB Message Notification callback function where you enable or disable the USB Device support based on the notification message coming from the **UDC driver**.
As mentioned, if the UDC driver does not monitor the presence of the Vbus, you need to use a GPIO pin to enable or disable the USB Device support.

25 Enabling the **USB Device support** when cable is connected

In the first lab you defined the ***vbus0*** pin that is active when the Vbus is present. You need to call the ***manage_usb_device()*** function when there is a transition from inactive to active and vice-versa, so you can use the input service, like you did for button0, but only if the **UDC driver** does not monitor Vbus.

HINT:

You need to get the code property of the Vbus pin and you can define it as ***VBUS_CODE*** using the **DT_PROP** macro and the **DT_NODELABEL** macro to refer to Vbus pin (***vbus0***). You need to define it before the ***input_cb()*** function.

In the ***input_cb()***, you can check if the ***managed_vbus*** variable is false and then check for the ***event->code:*** if it is related to Vbus pin (***VBUS_CODE***) then you need to check if the event value is active (***event->value == true***) and in that case enable the USB Device support using the ***manage_usb_device()*** function, passing ***true***, otherwise disabling it, passing ***false***.

Documentation is available here:

https://docs.zephyrproject.org/latest/build/dts/api/api.html#node-identifiers-and-helpers

Step 25 - Solution

26 Enabling the **USB Device support** if the cable is already connected

The previous step does not cover the case where the USB cable is already connected at Power-On so you need to take care about that in your code.

> HINT:
>
> You need to define a container for GPIO information to be used with the GPIO APIs; the container has type ***static const struct gpio_dt_spec*** and you can call it ***vbus_gpio***. You need to initialize it with the reference to ***vbus0*** pin that you can obtain with the macro **GPIO_DT_SPEC_GET()** where the node can be referenced using the **DT_NODELABEL()** macro and the property ***gpios***. Define it before the ***main()*** function.
>
> Now, in the ***main()*** function, just before the ***while(1)*** loop, you can check if the Vbus does not manage the Vbus (***!managed_vbus***) and then, if the ***vbus_gpio*** is high, you can call the ***manage_usb_device()*** function, passing ***true***. You can check the status of the GPIO pin with the function ***int gpio_pin_get_dt(const struct gpio_dt_spec *spec)*** that returns 1 if the pin is in active state or 0, if inactive.
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/latest/build/dts/api/api.html#node-identifiers-and-helpers
>
> https://docs.zephyrproject.org/latest/doxygen/html/group__gpio__interface.html

Step 26 - Solution

Finally, you added a new source file (**usb.c**) to the project so you need to update the list of target sources in **CMakeLists.txt** file.

27 Add the new source file to the **source file list**

Add ***src/usb.c*** to the ***target_source()*** list.

Step 27 - Solution

Build and program

You can now build the application using **west** in the **VS Code terminal**:

```
west build -b my_board -p always Lab
```

Copy

Then you can deploy the executable using **west** again:

```
west flash
```

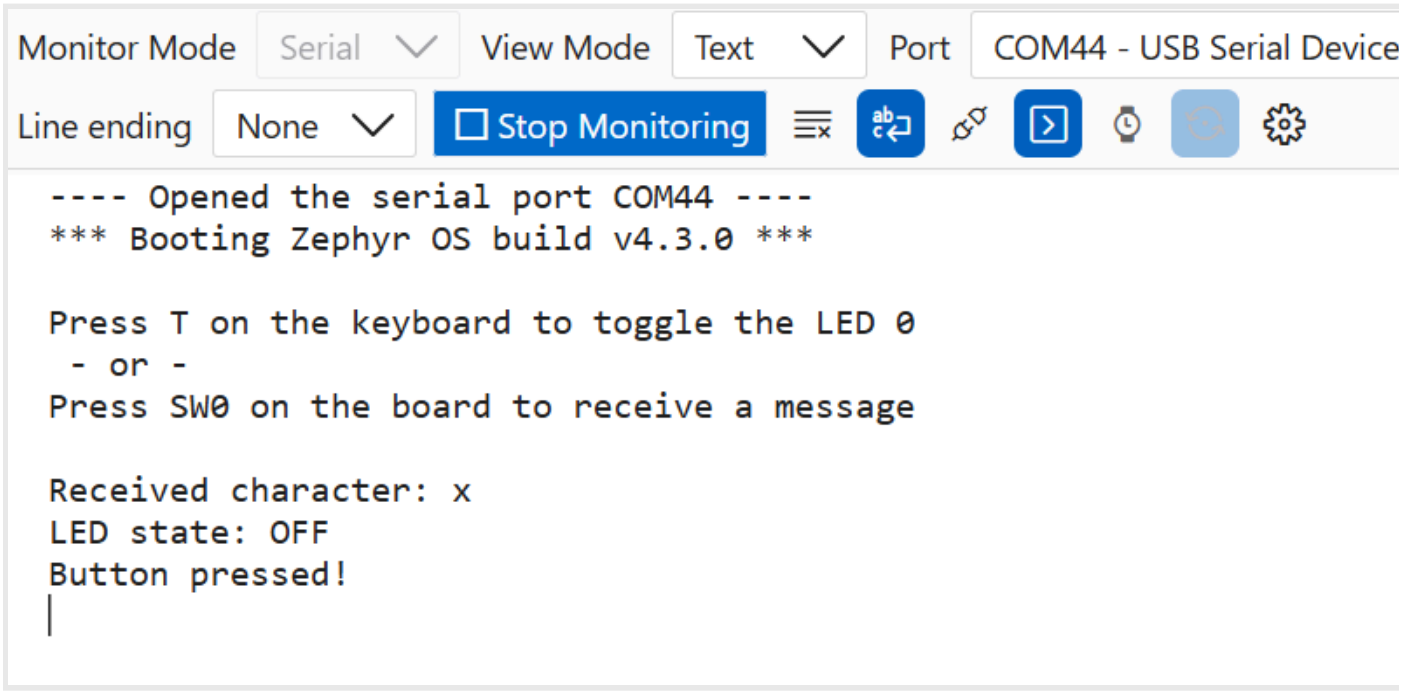
Copy

Connect the second USB cable, connected to "**TARGET USB**" port on the board, to an available USB port of the PC and use the **Serial Monitor** Extension, opening the **Port** corresponding to the **USB CDC-ACM** Device at **115200** Baude rate.

Results

You are going to have the same results as Lab 2, but this time you are using the USB CDC-ACM Device you created from scratch, not just redirecting the **Console**.

Like in Lab 2, if you press "**x**", then "**T**" and then the button **SW0**, you should see something like the following screenshot:



Summary

Congratulations! You have successfully configured the USB Device Stack to implement a **USB CDC-ACM Device** and communicated with the **Serial Monitor** running on the PC!

Continue to Lab 4...

Lab 3: Step 1 - Solution

Copy & Paste

```
CONFIG_SERIAL=y
CONFIG_UART_INTERRUPT_DRIVEN=y
CONFIG_UART_LINE_CTRL=y
CONFIG_HWINFO=y
```

Copy

Go back to Lab 3

Lab 3: Step 2 - Solution

Copy & Paste

usb.h

```
#include <zephyr/usb/usbd.h>
#include <zephyr/drivers/uart.h>
extern const struct device *cdc_dev;
```

usb.c

```
#include "usb.h"
const struct device *cdc_dev = DEVICE_DT_GET(DT_NODELABEL(cdc_acm_uart));
```

Go back to Lab 3

Lab 3: Step 3 - Solution

Copy & Paste

```
USBD_DEVICE_DEFINE(usbd_ctx, DEVICE_DT_GET(DT_NODELABEL(zephyr_udc0)), 0x04D8, 0x000A);
```

Copy

Go back to Lab 3

Lab 3: Step 4 - Solution

Copy & Paste

Copy

```
USBD_DESC_LANG_DEFINE(lang_desc);
USBD_DESC_MANUFACTURER_DEFINE(mfr_desc, "Microchip Technology Inc.");
USBD_DESC_PRODUCT_DEFINE(prod_desc, "Zephyr CDC-ACM");
USBD_DESC_SERIAL_NUMBER_DEFINE(sn_desc);
```

Go back to Lab 3

Lab 3: Step 5 - Solution

Copy & Paste

```
USBD_DESC_CONFIG_DEFINE(fs_cfg_desc, "FS Configuration");
USBD_CONFIGURATION_DEFINE(fs_config, USB_SCD_SELF_POWERED, 50, &fs_cfg_desc);
```

Copy

Go back to Lab 3

Lab 3: Step 6 - Solution

Copy & Paste

Copy

```
void usb_msg_callback(struct usbd_context *const ctx, const struct usbd_msg *const msg)
{
    if (usbd_can_detect_vbus(ctx)) {
        switch (msg->type)
        {
        case USBD_MSG_VBUS_READY:
            manage_usb_device(true);
            break;
        case USBD_MSG_VBUS_REMOVED:
            manage_usb_device(false);
            break;
        default:
            break;
        }
    }
}
```

Go back to Lab 3

Lab 3: Step 7 - Solution

Copy & Paste

usb.h

```
extern bool managed_vbus;
int init_usb_device(void);
```

Copy

usb.c

```
bool managed_vbus;
int init_usb_device(void)
{
    int err=0;

    return err;
}
```

Copy

Go back to Lab 3

Lab 3: Step 8 - Solution

Copy & Paste

Copy

```
err = usbd_add_descriptor(&usbd_ctx, &lang_desc);
if (err) {
    return err;
}
err = usbd_add_descriptor(&usbd_ctx, &mfr_desc);
if (err) {
    return err;
}
err = usbd_add_descriptor(&usbd_ctx, &prod_desc);
if (err) {
    return err;
}
err = usbd_add_descriptor(&usbd_ctx, &sn_desc);
if (err) {
    return err;
}
```

Go back to Lab 3

Lab 3: Step 9 - Solution

Copy & Paste

```
    err = usbd_add_configuration(&usbd_ctx, USBD_SPEED_FS, &fs_config);
    if (err) {
        return err;
    }
```

Copy

Go back to Lab 3

Lab 3: Step 10 - Solution

Copy & Paste

```
    err = usbd_register_class(&usbd_ctx, "cdc_acm_0", USBD_SPEED_FS, 1);
    if (err) {
        return err;
    }
```

Copy

Go back to Lab 3

Lab 3: Step 11 - Solution

Copy & Paste

```
err = usbd_msg_register_cb(&usbd_ctx, usb_msg_callback);
if (err) {
    return err;
}
```

Copy

Go back to Lab 3

Lab 3: Step 12 - Solution

Copy & Paste

```
err = usbd_init(&usbd_ctx);
if (err) {
    return err;
}
```

Copy

Go back to Lab 3

Lab 3: Step 13 - Solution

Copy & Paste

```
managed_vbus = usbd_can_detect_vbus(&usbd_ctx);
```

Copy

Go back to Lab 3

Lab 3: Step 14 - Solution

Copy & Paste

Copy

```
typedef struct {
    uint8_t buf[128];
    uint16_t length;
} vcp_buffer;
extern vcp_buffer data_to_send;
```

Go back to Lab 3

Lab 3: Step 15 - Solution

Copy & Paste

usb.c

```
void manage_usb_device(bool connected)
{

}
```

Copy

usb.h

```
void manage_usb_device(bool connected);
```

Copy

Go back to Lab 3

Lab 3: Step 16 - Solution

Copy & Paste

Copy

```
    if (connected) {
        usbd_enable(&usbd_ctx);
        uint32_t dtr = 0;
        while (!dtr) {
            uart_line_ctrl_get(cdc_dev, UART_LINE_CTRL_DTR, &dtr);
            k_sleep(K_MSEC(100));
        }
        data_to_send.length = sprintf(data_to_send.buf, "\nPress T on the keyboard to toggle the LED 0\n -
or -\nPress SW0 on the board to receive a message\n\n");
        uart_irq_tx_enable(cdc_dev);
    }
    else {
        usbd_disable(&usbd_ctx);
    }
```

Go back to Lab 3

Lab 3: Step 17 - Solution

Copy & Paste

```
#include "usb.h"
```

Copy

[Go back to Lab 3]

Lab 3: Step 18 - Solution

Copy & Paste

```
vcp_buffer data_to_send;
vcp_buffer received_data;
```

Copy

Go back to Lab 3

Lab 3: Step 19 - Solution

Copy & Paste

```
            data_to_send.length = sprintf(data_to_send.buf, "Button pressed!\n\r");
            uart_irq_tx_enable(cdc_dev);
```

Copy

Go back to Lab 3

Lab 3: Step 20 - Solution

Copy & Paste

Copy

```
void uart_irq_cb(const struct device *dev, void *user_data)
{
    static bool led_state = true;
    while (uart_irq_update(dev) && uart_irq_is_pending(dev)) {

    }
}
```

Go back to Lab 3

Lab 3: Step 21 - Solution

Copy & Paste

Copy

```
        if (uart_irq_rx_ready(dev)) {
            received_data.length = uart_fifo_read(dev, received_data.buf, sizeof(received_data.buf));
            if (received_data.length > 0) {
                if (received_data.buf[0] == 'T') {
                    gpio_pin_toggle_dt(&led);
                    led_state = !led_state;
                    data_to_send.length = sprintf(data_to_send.buf, "LED state: %s\n\r", led_state ? "ON" :
"OFF");
                }
                else {
                    data_to_send.length = sprintf(data_to_send.buf, "Received character: %c\n\r",
received_data.buf[0]);
                }
                uart_irq_tx_enable(dev);
            }
        }
```

Go back to Lab 3

Lab 3: Step 22 - Solution

Copy & Paste

Copy

```
        if (uart_irq_tx_ready(dev)) {
            uart_fifo_fill(dev, data_to_send.buf, data_to_send.length);
            uart_irq_tx_disable(dev);
        }
```

Go back to Lab 3

Lab 3: Step 23 - Solution

Copy & Paste

Replace the target source line with this one:

```
    if (!device_is_ready(cdc_dev)) {
        return 0;
    }
    ret = init_usb_device();
    if (ret <0) {
        return 0;
    }
```

Copy

Go back to Lab 3

Lab 3: Step 24 - Solution

Copy & Paste

```
uart_irq_callback_set(cdc_dev, uart_irq_cb);
uart_irq_rx_enable(cdc_dev);
```

Copy

Go back to Lab 3

Lab 3: Step 25 - Solution

Copy & Paste

Before ***input_cb()*** function:

```
#define VBUS_CODE DT_PROP(DT_NODELABEL(vbus0), zephyr_code)
```

Copy

Inside ***input_cb()*** function:

```
    if (!managed_vbus) {
        if (event->code == VBUS_CODE) {
            if (event->value) {
                manage_usb_device(true);
            }
            else {
                manage_usb_device(false);
            }
        }
    }
```

Copy

Go back to Lab 3

Lab 3: Step 26 - Solution

Copy & Paste

Before ***main()*** function:

```
static const struct gpio_dt_spec vbus_gpio = GPIO_DT_SPEC_GET(DT_NODELABEL(vbus0), gpios);
```

Copy

Inside ***main()*** function:

```
    if (!managed_vbus) {
        if (gpio_pin_get_dt(&vbus_gpio)) {
            manage_usb_device(true);
        }
    }
```

Copy

Go back to Lab 3

Lab 3: Step 27 - Solution

Copy & Paste

Replace the target sources line with this one:

```
target_sources(app PRIVATE src/main.c src/usb.c)
```

Copy

Go back to Lab 3

Lab 4 - Implement a simple Command-Response protocol

Purpose

In this lab you will learn how to configure the USB Device Stack to implement an USB HID Device and how to implement a simple Command-Response protocol to send and receive generic data.

Overview

You will reconfigure the USB Device Stack to implement a USB HID Device, creating the HID Report Descriptor structure, defining the HID Device Operation structure, registering them in the USB device context, then you will define the HID Device Operation callback functions, implementing the Command-Response protocol to have your device communicate with the provided PC application.

Procedure

This lab will continue from the previous lab. If you haven't finished the previous lab, please overwrite the content of **Lab** folder with the one you can find in the **Solution/lab3** folder.

You can copy the solution of Lab 3 over Lab folder using the provided batch file:

> **C:\MASTERs\26021_RTOS5\Copy_Lab3_Solution.bat**

or just run the following command from the Visual Studio Code terminal:

```
..\Copy_Lab3_Solution.bat
```

Copy

The basic configurations for the USB Device Stack you developed in Lab 3 can be reused to implement the USB HID Device, with some modifications.

1 Reconfigure the **USB node**

You need to reconfigure it to work a **USB HID Device**. You can do that in **app.overlay** file.

> HINT:
>
> Change the ***zephyr_udc0*** node label, commenting-ot the ***cdc_acm_uart*** child node and adding a child node labeled ***hid0*** with ***compatible*** property set to "***zephyr, hid-device***", ***protocol-code*** property set to ***"none"***, ***in-report-size*** set to ***64*** as well as the ***out-report-size***, then set both ***in-polling-period-us*** and ***out-polling-period-us*** to ***1000*** (1ms)
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/4.3.0/build/dts/intro-syntax-structure.html
>
> https://docs.zephyrproject.org/latest/build/dts/api/bindings/usb/zephyr%2Chid-device.html

Step 1 - Solution

2 Reconfigure the project

Then you need to modify the ***prj.conf*** file to include support for the USB HID Class.

> HINT:
>
> Comment-out the option related to CDC-ACM, Serial, UART and input and add the **kconfig option** to enable the support for the USB HID Class (**CONFIG_USBD_HID_SUPPORT**)
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/4.3.0/kconfig.html

Step 2 - Solution

3 Change the **Device Handle**

In the previous lab you defined a device handle for the USB CDC-ACM Device (***cdc_dev***).
In this lab you need to change it to define a device handle for the USB HID Device (***hid_dev***).
You can use the node label ***hid0*** instead of ***cdc_acm_uart*** to initialize the handle.
Like you did in Lab 3, declare it as external in **usb.h** and define it in **usb.c** file.
Comment-out the include directives of **UART driver** (<zephyr/drivers/uart.h> and include the header file to use the dedicated USB HID APIs in **usb.h** (zephyr/usb/class/usbd_hid.h)

Step 3 - Solution

The PC application provided for this lab is expecting a USB HID device with Product ID equal to 0x003F so you need to modify the definition of ***usbd_ctx*** accordingly.

4 Update **Product ID**

Change the **PID** from ***0x000A*** to ***0x003F*** in the ***usbd_ctx*** definition.

Step 4 - Solution

To easily find your USB Device in the list of connected devices, it is better to change the Product String.

5 Update **Product String Descriptor**

Change the ***prod_desc*** string from ***"Zephyr CDC-ACM"*** to ***"Zephyr vendor HID"***

Step 5 - Solution

You need to define a USB HID Report descriptor for the USB HID Device. The descriptor is an array of unsigned 8-bit integer that consists of specific items defined by the HID usage tables. An easy way to create that array is to ask **MPLAB AI Coding Assistant.**

6 Define the **USB HID Report Descriptor**

Ask to the **MPLAB AI Coding Assistant** to generate for you a HID report descriptor, named ***hid_report_descriptor***, for a vendor HID Report of 64 elements of 1 byte each, used for sending and receiving data. Copy the array in your code before the ***init_usb_device()*** function.

Step 6 - Solution

Other than the USB Device Stack Notification Messages, in a USB HID Device you need to handle the USB HID User callbacks.

7 Declare the **HID User Callbacks** functions and add them to an ***hid_device_ops*** structure

Declare the callbacks based on the device functionality.

HINT:

Declare ***hid_get_report_cb()***, ***hid_set_report_cb()***, ***hid_output_report_cb()*** and ***hid_input_report_done_cb()*** functions in **usb.h** file and define, in **usb.c** file, an ***hid_device_ops*** structure, named ***hid_ops***, assigning the functions to the corresponding members ***get_report***, ***set_report***, ***output_report*** and ***input_report_done***. Add it before ***init_usb_device()***.

Remember to use the same return type and parameters of the corresponding pointer to function, to avoid compile errors.

Documentation is available here:

https://docs.zephyrproject.org/4.3.0/doxygen/html/structhid__device__ops.html

Step 7 - Solution

You also need to modify the ***init_usb_device()*** function you defined in the previous lab.

8 Update the *init_usb_device()* function

Register the HID class, instead of the CDC-ACM Class, register the HID Report Descriptor and user callbacks, then set the code triple for an HID Device.

> HINT:
>
> Comment-out the CDC-ACM Class registration and use ***usbd_register_class()*** function to register the HID Class (***hid_0***) then you can use ***hid_device_register()*** to register in ***hid_dev*** the defined ***hid_report_descriptor***, with its size, and a pointer to the ***hid_ops*** structure.
>
> You can set the code triple of ***usbd_ctx*** to [***0***, ***0***, ***0***] with the ***usbd_device_set_code_triple()*** function. Remember that the USB peripheral supports ***USBD_SPEED_FS*** only.
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__hid__device.html
>
> https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__api.html

Step 8 - Solution

To complete the reconfiguration of the USB Device Stack to enumerate the device as USB HID Device, you also need to comment-out, in ***manage_usb_device()*** function, the check for the **DTR**, the string definition and the TX interrupt enable functions.

Next steps are removing the old USB CDC-ACM related code in **main.c** and adding the HID related code.

Comment-out the code property definition **SW0_CODE** and the related code in the ***input_cb()*** function, the two structures you used to send and receive data (***received_data*** and ***data_to_send***), the UART IRQ callback (***uart_irq_cb***).

In the ***main()*** function, comment-out the registration of the UART IRQ callback, the RX interrupt enable function; finally, you need to change the device handle you are checking in your code.

9 Update the **Device Handle**

Replace the device handle in the ***device_is_ready()*** function with ***hid_dev***.

Step 9 - Solution

In this lab the Input service is not used for the button because the PC App monitoring the state, and not the transitions, of the switch.

10 Define the container for **GPIO pin** information

Define, immediately before ***main()*** function, the container ***button*** of type ***static const struct gpio_dt_spec***,

> HINT:
>
> Initialize it using the macro **GPIO_DT_SPEC_GET()** with property ***gpios*** and use the macro **DT_NODELABEL()** to get the devicetree node identifier of ***button0***
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/latest/doxygen/html/group__devicetree-generic-id.html
>
> https://docs.zephyrproject.org/latest/doxygen/html/group__gpio__interface.html

Step 10 - Solution

11 Configure the **GPIO pin**

You need to configure the corresponding GPIO to be used as an input pin so you can get its status (pressed/not pressed).

> HINT:
>
> Check the readiness of the previously defined ***button*** using the ***gpio_is_ready_dt()*** function, returning ***0*** if it is not ready, then configure it to be a ***GPIO_INPUT*** with the ***gpio_pin_configure_dt()*** function, assigning the returned value to the variable ***ret*** and check if ***ret*** is less than zero, returning ***0*** if yes. The code needs to be placed in the ***main()*** function and you can put it immediately after the similar code for ***led*** container
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/latest/doxygen/html/group__gpio__interface.html

Step 11 - Solution

Now you need to define the user callbacks to handle the communication with the USB Host and implement the Command-Response protocol used by the PC application.

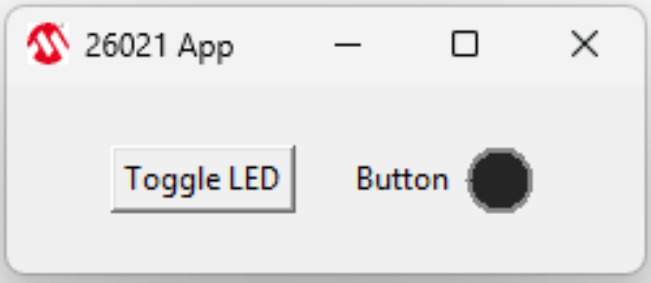
12 Dedine the **HID User Callbacks** functions for **Set Report** and **Get Report**

In this lab, the two required callback functions for ***hid_get_report*** and ***hid_set_report*** members of ***hid_ops*** are not used so you can define them as functions that just return ***-ENOTSUP***. Define them before ***main()*** function.

> HINT:
>
> You can copy the prototypes from the **usb.h** file and complete them.
>
> **-ENOTSUP** indicate that the feature is not supported.
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/4.3.0/doxygen/html/structhid__device__ops.html

Step 12 - Solution

The PC application ("**26021 app.exe**") is a simple application to toggle a LED and show the current status of a button on the board.



The app looks for a USB HID Device with VIP = 0x04D8 and PID = 0x003F and, if the Device is connected to the Host, it enables the "**Toggle LED**" button. Pressing that button, the app sends an output report with the toggle LED command, setting the first byte of the output report at ***0x80***; no answer from the device is expected for this command.
The app also periodically send an output report with get button status command, setting the first byte of output report at ***0x81***, followed by a request of an input report, expecting a reply where the first byte of the report is the echo of the command (***0x81***) and the second byte is ***0***, if the button is pressed, ***1*** if it is not pressed; the app change the color of the "**Button**" image based on the answer.

First, you need to define the callback that handles the output report.

13 Define the **HID User Callbacks** functions for **Output Report**

Define the ***hid_output_report_cb()*** function, you already declared in **usb.h** file, just before ***main()*** function. In the function, define a global variable, a boolean ***led_state***, initialized at true, to store the status of the LED, and a local ***static*** variable, an unsigned 8-bit integer array of 64 bytes named ***input_report***, that will be used to provide the answer to the "**Get button status**" command. Check if the size of the received buffer (***len***) is greater than ***0*** and if the pointer to the buffer (***buf***) is not pointing to NULL, then add a switch-case statement looking for the value of the first byte of the buffer: the two cases you need to implement are ***buf[0] = 0x80*** and ***buf[0] = 81***. You will complete the two cases in the following steps.

Step 13 - Solution

14 Implement the **Toggle LED** command

The first command you need to implement is toggle LED command (***case 0x80***) and you can use the ***led_state*** variable to keep track of LED state.

> HINT:
>
> You can toggle the LED using the ***gpio_pin_toggle_dt()*** function (returning immediately if the function does not return 0), then you need to negate the ***led_state*** variable to keep track of LED state.
>
> Documentation is available here:
>
> https://docs.zephyrproject.org/latest/doxygen/html/group__gpio__interface.html

Step 14 - Solution

15 Implement the **Get Button Status** command

Then you need to implement the get button status command (***case 0x81***), where you need to prepare and send the response to the command.

> HINT:
>
> You can use the ***input_report*** array to prepare the response, setting the first bite at ***0x81*** and the second byte at:
>
> - ***0***: SW0 is pressed
> - ***1***: SW0 is pressed
>
> You can get the status of the SW0 button using the ***gpio_pin_get_dt()*** function that returns true if the pin is active (button pressed) or false if the pin is inactive (button not pressed).
>
> Once the array is filled with the response, you can send it to the host submitting the request with ***hid_device_submit_report()*** function.
>
> Documentation is available here:
>
> **https://docs.zephyrproject.org/latest/doxygen/html/group__gpio__interface.html**
>
> **https://docs.zephyrproject.org/4.3.0/doxygen/html/group__usbd__hid__device.html**

Step 15 - Solution

Last user callback you need to define is the one required for ***input_report_done*** member, that is called when the input report has been received by the host. You can use this function to report, via UART Console, the status of the LED and button.

16 Define the **HID User Callbacks** functions for **Input Report Done**

Define the ***hid_input_report_done_cb()*** function and use ***printk()*** to send the string containing the state of the LED (similar to the ***printf()*** you can find in the ***while(1)*** loop of the ***main()*** function); do the same to send the string containing the state of the button you reported with the input report (similar to the previous one but checking ***report[1]*** to decide if you need to print ***"Button is pressed\n"*** or ***"Button is not pressed\n"***).
If you add, at the end of the second string, "***\e[F\e[F***" the text in the terminal will update itself, instead of just adding new lines.

Step 16 - Solution

Build and program

You can now build the application using **west** in the **VS Code terminal**:

```
west build -b my_board -p always Lab
```

Copy

Then you can deploy the executable using **west** again:

```
west flash
```

Copy

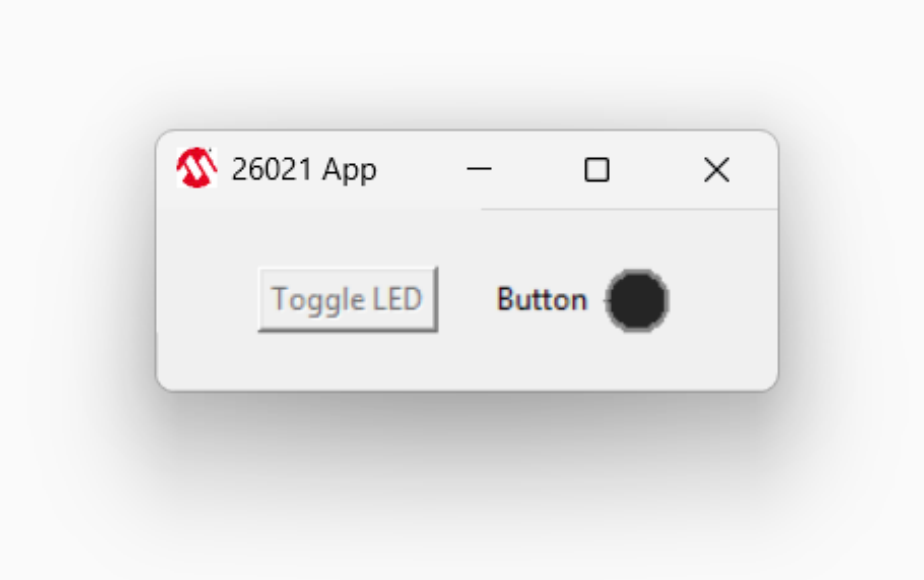
Connect the second USB cable, that is connected to "**TARGET USB**" port on the board, to an available USB port of the PC and launch the PC application from the VS Code terminal executing the following command block:

```
& {
    cd .\App\
    & '.\26021 App.exe'
    cd ..
}
```

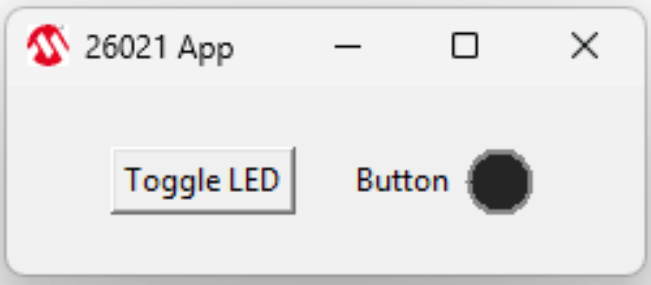
Copy

Results

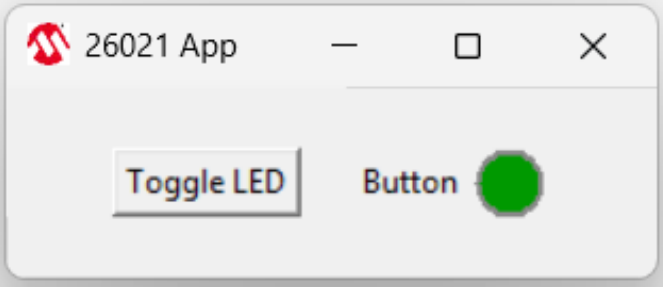
If the board is not connected through the "TARGET USB" port, the USB HID Device is not connected, and the app keep the "Toggle LED" button disabled:



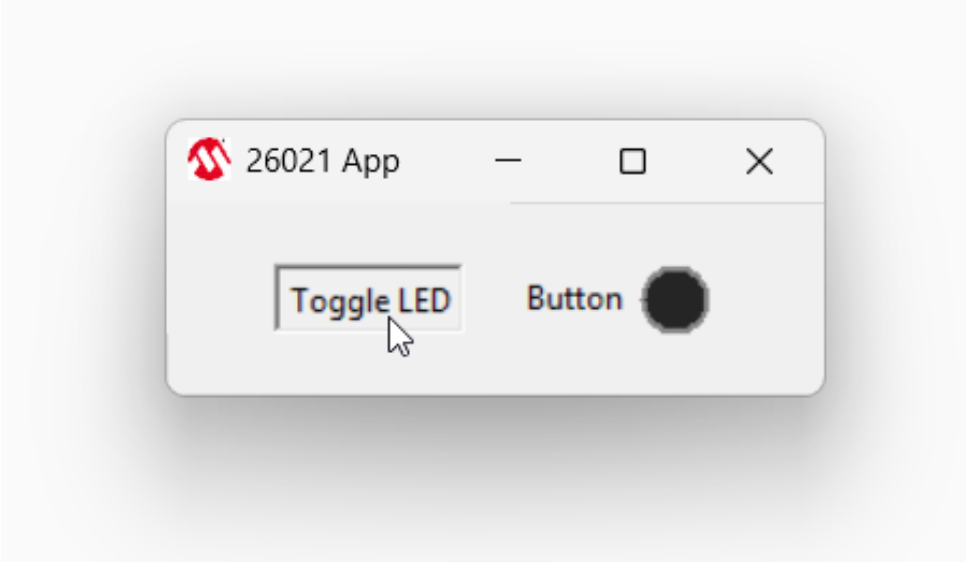
When you connect the board, the USB HID Device is enumerated and the app enables the "Toggle LED" button. If the button SW0 is not pressed, the button indicator is off (black):



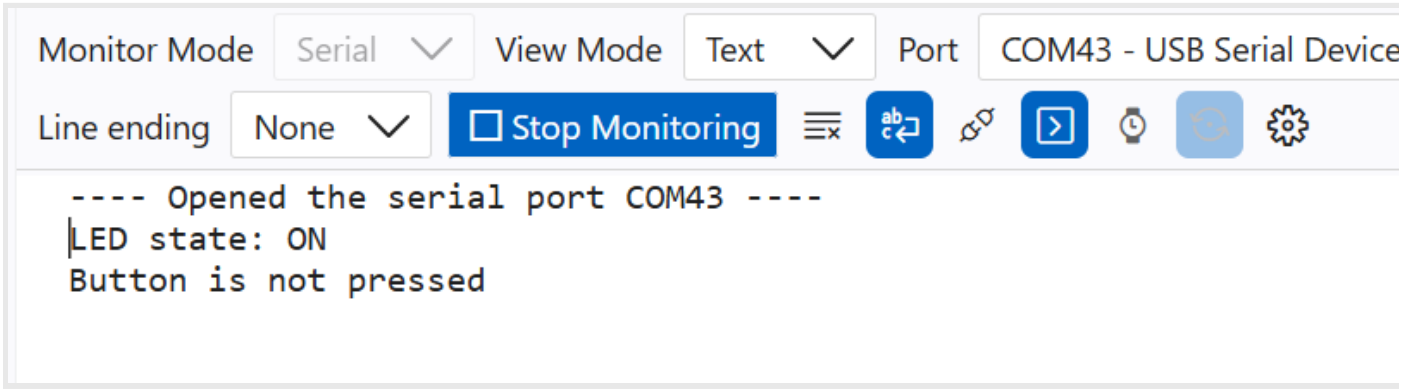
If you press the SW0 button on the board, the button indicator is on (green):



If you press the "Toggle LED" button on the app, you toggle the status of LED0 on the board:



If you use the **Serial Monitor** Extension, opening the **Port** corresponding to the **EDBG** serial bridge at **115200** baud rate, you see the messages related to the status of the LED and the status of the button:



Summary

Congratulations! You have successfully configured the USB Device Stack to implement a **USB HID Device** and communicated with the PC application implementing a simple Command-Response protocol!

Go back to Lab Manual Home...

Lab 4: Step 1 - Solution

Copy & Paste

Copy

```
hid0: hid0 {
    compatible = "zephyr,hid-device";
    protocol-code = "none";
    in-report-size = <64>;
    out-report-size = <64>;
    in-polling-period-us = <1000>;
    out-polling-period-us = <1000>;
};
```

Go back to Lab 4

Lab 4: Step 2 - Solution

Copy & Paste

```
CONFIG_USBD_HID_SUPPORT=y
```

Copy

Go back to Lab 4

Lab 4: Step 3 - Solution

Copy & Paste

usb.h

```
#include <zephyr/usb/class/usbd_hid.h>
extern const struct device *hid_dev;
```

Copy

usb.c

```
const struct device *hid_dev = DEVICE_DT_GET(DT_NODELABEL(hid0));
```

Copy

Go back to Lab 4

Lab 4: Step 4 - Solution

Copy & Paste

Replace the definition with this one:

```
USBD_DEVICE_DEFINE(usbd_ctx, DEVICE_DT_GET(DT_NODELABEL(zephyr_udc0)), 0x04D8, 0x003F);
```

Copy

Go back to Lab 4

Lab 4: Step 5 - Solution

Copy & Paste

Replace the definition with this one:

```
USBD_DESC_PRODUCT_DEFINE(prod_desc, "Zephyr vendor HID");
```

Copy

Go back to Lab 4

Lab 4: Step 6 - Solution

Copy & Paste

Copy

```
const uint8_t hid_report_descriptor[] = {
    0x06, 0x00, 0xFF,      // Usage Page (Vendor Defined 0xFF00)
    0x09, 0x01,            // Usage (Vendor Usage 1)
    0xA1, 0x01,            // Collection (Application)

    // Input report (device → host): 64 bytes
    0x15, 0x00,            //   Logical Minimum (0)
    0x25, 0xFF,            //   Logical Maximum (255)
    0x75, 0x08,            //   Report Size (8 bits/byte)
    0x95, 0x40,            //   Report Count (64 bytes)
    0x09, 0x01,            //   Usage (Vendor Usage 1)
    0x81, 0x02,            //   Input (Data, Variable, Absolute)

    // Output report (host → device): 64 bytes
    0x15, 0x00,            //   Logical Minimum (0)
    0x25, 0xFF,            //   Logical Maximum (255)
    0x75, 0x08,            //   Report Size (8 bits/byte)
    0x95, 0x40,            //   Report Count (64 bytes)
    0x09, 0x01,            //   Usage (Vendor Usage 1)
    0x91, 0x02,            //   Output (Data, Variable, Absolute)

    0xC0                   // End Collection
};
```

Go back to Lab 4

Lab 4: Step 7 - Solution

Copy & Paste

usb.h

```
int hid_get_report_cb(const struct device *dev, const uint8_t type, const uint8_t id, const uint16_t len,
uint8_t *const buf);
int hid_set_report_cb(const struct device *dev, const uint8_t type, const uint8_t id, const uint16_t len,
const uint8_t *const buf);
void hid_output_report_cb(const struct device *dev, const uint16_t len, const uint8_t *const buf);
void hid_input_report_done_cb(const struct device *dev, const uint8_t *const report);
```

usb.c

```
struct hid_device_ops hid_ops = {
    .get_report = hid_get_report_cb,
    .set_report = hid_set_report_cb,
    .input_report_done = hid_input_report_done_cb,
    .output_report = hid_output_report_cb,
};
```

Go back to Lab 4

Lab 4: Step 8 - Solution

Copy & Paste

Copy

```
err = usbd_register_class(&usbd_ctx, "hid_0", USBD_SPEED_FS, 1);
if (err) {
    return err;
}
err = hid_device_register(hid_dev, hid_report_descriptor, sizeof(hid_report_descriptor), &hid_ops);
if (err) {
    return err;
}
err = usbd_device_set_code_triple(&usbd_ctx, USBD_SPEED_FS, 0, 0, 0);
if (err) {
    return err;
}
```

Go back to Lab 4

Lab 4: Step 9 - Solution

Copy & Paste

Replace the check of readiness of cdc_dev with this one:

```
    if (!device_is_ready(hid_dev)) {
        return 0;
    }
```

Copy

Go back to Lab 4

Lab 4: Step 10 - Solution

Copy & Paste

```
static const struct gpio_dt_spec button = GPIO_DT_SPEC_GET(DT_NODELABEL(button0), gpios);
```

Copy

Go back to Lab 4

Lab 4: Step 11 - Solution

Copy & Paste

Copy

```
if (!gpio_is_ready_dt(&button)) {
    return 0;
}
ret = gpio_pin_configure_dt(&button, GPIO_INPUT);
if (ret < 0) {
    return 0;
}
```

Go back to Lab 4

Lab 4: Step 12 - Solution

Copy & Paste

Copy

```
int hid_get_report_cb(const struct device *dev, const uint8_t type, const uint8_t id, const uint16_t len,
uint8_t *const buf)
{
 return -ENOTSUP;
}
int hid_set_report_cb(const struct device *dev, const uint8_t type, const uint8_t id, const uint16_t len,
const uint8_t *const buf)
{
 return -ENOTSUP;
}
```

Go back to Lab 4

Lab 4: Step 13 - Solution

Copy & Paste

Copy

```
bool led_state = true;
void hid_output_report_cb(const struct device *dev, const uint16_t len, const uint8_t *const buf)
{
    static uint8_t input_report[64];
    if (len > 0 && buf != NULL) {
        switch (buf[0]) {
            case 0x80: {


                break;
            }
            case 0x81: {


                break;
            }
        }
    }
}
```

Go back to Lab 4

Lab 4: Step 14 - Solution

Copy & Paste

Copy

```
if(gpio_pin_toggle_dt(&led) != 0) {
    return;
}
led_state = !led_state;
```

Go back to Lab 4

Lab 4: Step 15 - Solution

Copy & Paste

Copy

```
input_report[0] = buf[0];
if (gpio_pin_get_dt(&button)) {
    input_report[1] = 0;
}
else {
    input_report[1] = 1;
}
hid_device_submit_report(dev, 64, input_report);
```

Go back to Lab 4

Lab 4: Step 16 - Solution

Copy & Paste

Copy

```
void hid_input_report_done_cb(const struct device *dev, const uint8_t *const report)
{
    printk("LED state: %s\n", led_state ? "ON " : "OFF");
    printk("Button is %s\n\e[F\e[F", report[1] ? "not pressed" : "pressed    ");
}
```

Go back to Lab 4

Supplement A - Labs files and Required Software Setup

Labs files Setup

You can install labs files on your PC opening a Windows PowerShell and executing the following command:

```
powershell -ExecutionPolicy Bypass -Command "IEX (Invoke-RestMethod
'https://raw.githubusercontent.com/gcolombo-
mchp/26021_RTOS5/refs/heads/main/Scripts/install_26021_RTOS5.ps1')"
```

Copy

The script checks if the required software is installed before installing the labs files. If the process ends because a required software is missing, you can use the following section to install it.

Required Software Setup

You can install the required software by downloading the installation files from their website, or you can run a script that has been created to check if a required software is missing and install it.

Open a Windows PowerShell with administrator rights (right click on its icon and select "Run as administrator") then execute the following command:

```
powershell -ExecutionPolicy Bypass -Command "IEX (Invoke-RestMethod
'https://raw.githubusercontent.com/gcolombo-
mchp/26021_RTOS5/refs/heads/main/Scripts/Install_Required_Software_26021_RTOS5.ps1')"
```

Copy

If the script fails, you can install the missing packages manually, opening a Windows PowerShell and using **winget** commands:

- **Visual Studio Code:**

```
winget install Microsoft.VisualStudioCode
```

Copy

You will need to create a profile with the following Extensions installed:

- Python
- C/C++ Extension Pack
- Serial Monitor
- Cortex-Debug
- MPLAB AI Coding Assistant

- **Python v3.12:**

```
winget install Python.Python.3.12
```

Copy

- **CMake:**

```
winget install Kitware.CMake
```

Copy

- **Ninja:**

```
winget install Ninja-build.Ninja
```

Copy

- **Gperf:**

```
winget install oss-winget.gperf
```

Copy

- **Git:**

```
winget install Git.Git
```

Copy

- **Device Tree Compiler:**

```
winget install oss-winget.dtc
```

Copy

- **Wget:**

```
winget install JernejSimoncic.Wget
```

Copy

- **7-Zip:**

```
winget install 7zip.7zip
```

Copy

- **openOCD:**

```
winget install xpack-dev-tools.openocd-xpack
```

Copy

The **Zephyr SDK v0.17.4** cannot be installed using **winget** so you need to download, extract and setup it with a sequence of commands that requires **Wget** and **7-Zip** already available (if you just installed them, you might need to close and reopen Windows PowerShell to update the environment path):

Copy

```
& {
    cd $home
    & cmd.exe /c wget https://github.com/zephyrproject-rtos/sdk-ng/releases/download/v0.17.4/zephyr-sdk-
0.17.4_windows-x86_64.7z
    7z x zephyr-sdk-0.17.4_windows-x86_64.7z -aoa
    cd zephyr-sdk-0.17.4
    & cmd.exe /c setup.cmd
    cd ..
    Remove-Item -Force ".\zephyr-sdk-0.17.4_windows-x86_64.7z"
}
```

Go back to Lab Manual Home...